

Reviewer Pack — Minimal Order of Modular Functions Bounds Tree-like Resoluti...

Entry b44575be8ecb · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-30 01:14:27 UTC

Minimal Order of Modular Functions Bounds Tree-like Resolution Proof Width

Entry ID: b44575be8ecb **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-30 01:14:27 UTC

1. Conjecture

Field A (mathematical branch): Modular Function Theory **Field B** (complexity object): Boolean Function Complexity

Statement:

For each CNF φ with n variables, let f_φ be the minimal order of a modular function that can represent the truth table of φ over any finite field. Then, the tree-like resolution proof width of φ is $\Theta(2^{(n \cdot f_\varphi)})$.

Rationale (proposer's reasoning):

Modular functions are a generalization of polynomials and have applications in number theory and algebraic geometry. Their minimal order could provide insights into the complexity of representing Boolean functions, potentially leading to new lower bounds for resolution proofs.

Taxonomy category: cg_kw_andreev (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 72e6fc3e9d744166

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all generated CNFs, the calculated tree-like resolution proof width is within a factor of $2^{(n \cdot f_\varphi)}$ with an 80% support fraction and mean difference ≤ 3 across seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "modular function theory" AND "Boolean function complexity" AND "tree-like resolution proof width" - "minimal order of modular functions" AND CNF AND "truth table representation" - "CNF truth table" AND modular function AND "exponential bound tree-like resolution"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        cnf = []
        for _ in range(2**n):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            if not any(x == 0 for x in clause):
                cnf.append(clause)
        return cnf

    def truth_table(cnf):
        n = len(cnf[0])
        table = []
        for assignment in product([-1, 1], repeat=n):
            if all(any(x * assignment[abs(lit)-1] >= 0 for x in clause) for clause in cnf):
                table.append(assignment)
        return table

    def min_modular_function_order(table):
        n = len(table[0])
        max_value = max(abs(val) for row in table for val in row)
        order = 0
        while True:
            found = False
            for i in range(n):
                if all(row[i] * (val + 1) >= 0 for row in table):
                    found = True
                    break
            if found:
                order += 1
        return order
```

```

        break
    if not found:
        return order
    order += 1

def tree_like_resolution_width(cnf, table):
    n = len(table[0])
    width = 2**n
    for assignment in table:
        clause = [x * assignment[abs(lit)-1] >= 0 for lit in cnf]
        if all(clause):
            width = min(width, sum(1 for x in assignment if x != 0))
    return width

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    cnf = generate_cnf(n)
    table = truth_table(cnf)
    order = min_modular_function_order(table)
    width = tree_like_resolution_width(cnf, table)

    if order == 0 or width == 0:
        return {
            "metric_name": "tree_like_resolution_width",
            "metric_value": None,
            "instances_tested": len(table),
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    results.append({
        "n": n,
        "order": order,
        "width": width
    })

mean_width = sum(res["width"] for res in results) / len(results)
std_width = math.sqrt(sum((res["width"] - mean_width)**2 for res in results) / len(results))
support_fraction = sum(1 for res in results if abs(res["width"] - 2**(n * res["order"])) <= 3)

return {
    "metric_name": "tree_like_resolution_width",
    "metric_value": mean_width,
    "instances_tested": sum(res["instances_tested"] for res in results),
    "n_max": max(res["n"] for res in results),
    "conjecture_holds": support_fraction >= 0.8,
    "counterexample": "" if support_fraction >= 0.8 else f"n={results[0]['n']}, order={results[0]['order']}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]

```


11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	19152
2	preregistration	ollama_remote	glm4:latest	0	0	12178
3	novelty	ollama_remote	glm4:latest	0	0	9004
4	novelty	ollama_remote	glm4:latest	0	0	8672
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14184
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18732
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23744
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19309
9	judge	ollama_remote	glm4:latest	0	0	15004

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 139980 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/b44575be8ecb.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b44575be8ecb.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/b44575be8ecb.tar.gz` (if generated)